

Modelagem Geométrica: Representações Celulares e Esferas Geodésicas

Relatório de Implementação

2 de Fevereiro de 2026

Abstract

Este relatório apresenta a implementação de representações celulares uniformes e adaptativas para formas geométricas bidimensionais (discos e parábolas), bem como a construção de esferas geodésicas por refinamento de sólidos platônicos. Demonstramos formalmente os fundamentos matemáticos de cada técnica e descrevemos os algoritmos utilizados. A implementação pode ser encontrada em <https://github.com/rick0110/projects-computer-graphics-summer-2026>.
IMPA

1 Representações Celulares Uniformes

1.1 Fundamentação Matemática

Definição 1.1 (Partição Uniforme). Seja $\Omega = [a, b] \times [c, d] \subset \mathbb{R}^2$ um retângulo. Uma *partição uniforme* de Ω em $n \times n$ células é a coleção $\mathcal{P} = \{C_{i,j}\}_{i,j=1}^n$ onde:

$$C_{i,j} = [x_i, x_{i+1}] \times [y_j, y_{j+1}], \quad x_i = a + i \cdot \Delta x, \quad y_j = c + j \cdot \Delta y$$

com $\Delta x = \frac{b-a}{n}$ e $\Delta y = \frac{d-c}{n}$.

Definição 1.2 (Classificação de Células para Disco). Seja $D = \{(x, y) \in \mathbb{R}^2 : (x - c_x)^2 + (y - c_y)^2 \leq r^2\}$ um disco com centro (c_x, c_y) e raio r . Para cada célula $C = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$, definimos:

$$\sigma(C) = \begin{cases} 0 & \text{se } C \cap D = \emptyset \quad (\text{exterior}) \\ 1 & \text{se } C \subseteq D \quad (\text{interior}) \\ 2 & \text{se } C \cap \partial D \neq \emptyset \quad (\text{interseção}) \end{cases}$$

Teorema 1.3 (Critério de Classificação para Disco). *Seja $C = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ uma célula retangular e D um disco de centro (c_x, c_y) e raio r . Então:*

(i) $\sigma(C) = 0$ se e somente se $d_{\min}^2 > r^2$, onde $d_{\min}$ é a distância do centro do disco ao ponto mais próximo de C .

(ii) $\sigma(C) = 1$ se e somente se $d_k^2 < r^2$ para todos os vértices v_k de C .

(iii) $\sigma(C) = 2$ caso contrário.

Proof. (i) O ponto mais próximo de C ao centro (c_x, c_y) é dado por:

$$p^* = (\text{clamp}(c_x, x_{\min}, x_{\max}), \text{clamp}(c_y, y_{\min}, y_{\max}))$$

onde $\text{clamp}(z, a, b) = \max(a, \min(z, b))$. Se $\|p^* - (c_x, c_y)\|^2 > r^2$, então todo ponto de C está a distância maior que r do centro, logo $C \cap D = \emptyset$.

(ii) Se todos os vértices $v_k \in \{(x_{\min}, y_{\min}), (x_{\max}, y_{\min}), (x_{\max}, y_{\max}), (x_{\min}, y_{\max})\}$ satisfazem $\|v_k - (c_x, c_y)\|^2 < r^2$, então pela convexidade de D e C , temos $C \subset D$.

(iii) Segue por exclusão dos casos anteriores. □ □

Definição 1.4 (Classificação para Região Parabólica). Seja $R = \{(x, y) \in \mathbb{R}^2 : y \geq ax^2 + bx + c\}$. Para uma célula C :

$$\sigma(C) = \begin{cases} 0 & \text{se } C \cap R = \emptyset \\ 1 & \text{se } C \subseteq R \\ 2 & \text{se } C \cap \partial R \neq \emptyset \end{cases}$$

Teorema 1.5 (Critério de Interseção Parabola-Retângulo). A célula C intersecta a fronteira $\partial R : y = ax^2 + bx + c$ se e somente se pelo menos uma das seguintes condições é satisfeita:

- (i) A parábola cruza uma aresta vertical: $y_{\min} \leq ax_{\min}^2 + bx_{\min} + c \leq y_{\max}$ ou análogo para $x_{\max}$.
- (ii) A parábola cruza uma aresta horizontal: existem soluções $x \in [x_{\min}, x_{\max}]$ de $ax^2 + bx + (c - y_k) = 0$ para $y_k \in \{y_{\min}, y_{\max}\}$.

1.2 Implementação

A classificação de uma célula C pode ser determinada da seguinte forma:

- $\sigma(C) = 0$ se todos os vértices de C estiverem abaixo da parábola, pois a região acima da parábola é convexa se $a > 0$.
- $\sigma(C) = 2$ se houver interseção das arestas da célula com a parábola, conforme as condições acima.
- $\sigma(C) = 1$ se as condições anteriores não forem satisfeitas.

A implementação utiliza **PyTorch** com **vmap** para vetorização massiva das operações. As células são processadas em paralelo, explorando aceleração GPU quando disponível:

$$\text{Grid} : \mathbb{R}^{n \times n \times 4 \times 2} \xrightarrow{\text{vmap}} \mathbb{Z}_3^{n \times n} \quad (1)$$

2 Representações Celulares Adaptativas (Quadtree)

2.1 Fundamentação Matemática

Definição 2.1 (Quadtree). Uma *quadtree* T sobre $\Omega \subset \mathbb{R}^2$ é uma árvore quaternária onde cada nó v representa uma célula retangular C_v , e seus quatro filhos (se existirem) representam a subdivisão de C_v em quatro sub-células de igual tamanho.

Teorema 2.2 (Complexidade Espacial Adaptativa). Seja ∂S a fronteira de uma forma S com comprimento de arco L . Para uma quadtree de profundidade máxima d sobre $\Omega = [0, 1]^2$, o número de células na fronteira é $O(L \cdot 2^d)$, enquanto células internas/externas requerem $O(1)$ nós (não são subdivididas).

Proof. Cada célula de fronteira no nível d tem lado $\ell = 2^{-d}$. A fronteira ∂S atravessa no máximo $O(L/\ell) = O(L \cdot 2^d)$ células. Células completamente internas ou externas não são subdivididas pelo critério adaptativo. $\square$ $\square$

Algoritmo de Construção Adaptativa:

1. BUILD(Ω , forma S , profundidade máxima d):

- $\sigma \leftarrow \text{CLASSIFY}(\Omega, S)$
- raiz $\leftarrow \text{NODE}(\Omega, \sigma)$
- Se $\sigma = 2$ e profundidade $< d$: SUBDIVIDE(raiz, S , d)

2. SUBDIVIDE(nó v , forma S , d):

- Para cada quadrante Q de C_v :
- $\sigma_Q \leftarrow \text{CLASSIFY}(Q, S)$
- filho $\leftarrow \text{NODE}(Q, \sigma_Q)$
- Se $\sigma_Q = 2$ e profundidade(filho) $< d$: SUBDIVIDE(filho, S , d)

2.2 Poda da Árvore

Proposição 2.3 (Critério de Poda). *Se todos os filhos de um nó v possuem status idêntico $\sigma \in \{0, 1\}$, então os filhos podem ser removidos e v recebe status σ .*

3 Esfera Geodésica por Refinamento

3.1 Fundamentação Matemática

Definição 3.1 (Projeção Radial na Esfera). Seja $\mathbb{S}^2 = \{x \in \mathbb{R}^3 : \|x\| = 1\}$ a esfera unitária. A *projeção radial* $\pi : \mathbb{R}^3 \setminus \{0\} \rightarrow \mathbb{S}^2$ é definida por:

$$\pi(x) = \frac{x}{\|x\|}$$

Teorema 3.2 (Preservação de Estrutura Triangular). *Seja $T = \{v_1, v_2, v_3\}$ um triângulo com vértices em $\mathbb{S}^2$. O refinamento por subdivisão mediana seguido de projeção radial produz quatro triângulos geodésicos bem-definidos.*

Proof. Dado um triângulo esférico T com vértices $v_1, v_2, v_3 \in \mathbb{S}^2$:

Passo 1: Calculamos os pontos médios das arestas:

$$m_{12} = \frac{v_1 + v_2}{2}, \quad m_{23} = \frac{v_2 + v_3}{2}, \quad m_{31} = \frac{v_3 + v_1}{2}$$

Passo 2: Projetamos na esfera:

$$m'_{ij} = \pi(m_{ij}) = \frac{m_{ij}}{\|m_{ij}\|}$$

Passo 3: Os quatro novos triângulos são:

$$T_1 = (v_1, m'_{12}, m'_{31}), \quad T_2 = (v_2, m'_{23}, m'_{12}), \quad T_3 = (v_3, m'_{31}, m'_{23}), \quad T_4 = (m'_{12}, m'_{23}, m'_{31})$$

Cada triângulo tem vértices em $\mathbb{S}^2$ por construção. □ □

Proposição 3.3 (Dinâmica de Crescimento da Malha). *Seja $\mathcal{M}_0$ uma malha triangular fechada (homeomorfa a $\mathbb{S}^2$) com F_0 faces, A_0 arestas e V_0 vértices. Após k iterações do algoritmo de refinamento por subdivisão mediana, as quantidades de elementos são:*

$$F_k = 4^k F_0, \quad A_k = 4^k A_0, \quad V_k = 2 + 4^k (V_0 - 2)$$

Proof. (i) **Faces (F_k):** O algoritmo subdivide cada face triangular T em quatro novas faces conectando os pontos médios das arestas. Logo, o número de faces quadruplica a cada iteração:

$$F_k = 4 \cdot F_{k-1} \implies F_k = 4^k F_0$$

(ii) **Arestas (A_k):** Demonstramos a recorrência $A_k = 4A_{k-1}$ pela construção geométrica. Cada aresta antiga é dividida em duas (gerando $2A_{k-1}$ arestas), e em cada uma das F_{k-1} faces inserimos 3 novas arestas internas:

$$A_k = 2A_{k-1} + 3F_{k-1}$$

Como a malha é triangular, vale $3F_{k-1} = 2A_{k-1}$. Substituindo:

$$A_k = 2A_{k-1} + 3 \cdot \frac{2A_{k-1}}{3} = 4A_{k-1} \implies A_k = 4^k A_0$$

(iii) **Vértices (V_k):** Utilizamos a Característica de Euler para superfícies fechadas orientáveis de gênero 0:

$$V_k - A_k + F_k = \chi(\mathbb{S}^2) = 2$$

Isolando V_k e substituindo os resultados de (i) e (ii):

$$V_k = 2 + A_k - F_k = 2 + 4^k A_0 - 4^k F_0 = 2 + 4^k (A_0 - F_0)$$

Da Característica de Euler na malha inicial, $V_0 - A_0 + F_0 = 2$, temos $A_0 - F_0 = V_0 - 2$. Portanto:

$$V_k = 2 + 4^k (V_0 - 2) \quad \square$$

$\square$

Corolário 3.4. *Para o icosaedro ($V_0 = 12$, $A_0 = 30$, $F_0 = 20$), com $k = 8$ refinamentos:*

$$F_8 = 4^8 \cdot 20 = 1.310.720 \text{ faces}, \quad V_8 = 2 + 4^8 \cdot 10 = 655.362 \text{ vértices}$$

3.2 Formato OBJ

A malha resultante é exportada no formato Wavefront OBJ:

- **v x y z:** vértice com coordenadas normalizadas ($\|v\| = 1$)
- **f i j k:** face triangular com índices de vértices (1-indexado)

4 Resultados e Dificuldades

4.1 Resultados

- **Uniformes:** Grades processadas em paralelo com suporte para GPU.
- **Adaptativas:** Quadrees construídas recursivamente analisando apenas células de fronteira, sendo mais eficiente em CPU. Além disso, pode-se ver a construção recursiva por meio de animação.
- **Esfera:** Refinamento do icosaedro e geração de arquivo obj. Resultados de visualização foram aceitáveis.

4.2 Dificuldades Encontradas

1. **Divisão adaptativa e uniforme:** Adaptar algoritmos para `vmap` do PyTorch exigiu evitar alguns tipos de operações e mudar a estrutura de fluxo do algoritmo para que houvesse apenas operações vetoriais e que permitisse um fluxo idêntico para que se pudesse dar uma única instrução para várias threads. Além disso, como o método de classificação de células em parâbloas não estava nos slides, tive que pensar um pouco antes de implementar (tive problemas para debugar o código, pois a minha primeira intuição de classificação – sem pensar na lógica matemática – estava errada).
2. **Unificação de Vértices:** O refinamento ingênuo cria vértices duplicados; uma estrutura de hash – sugerida no material – foi necessária para malhas maiores (porém tive que pensar um pouco sobre como implementar).

5 Figuras

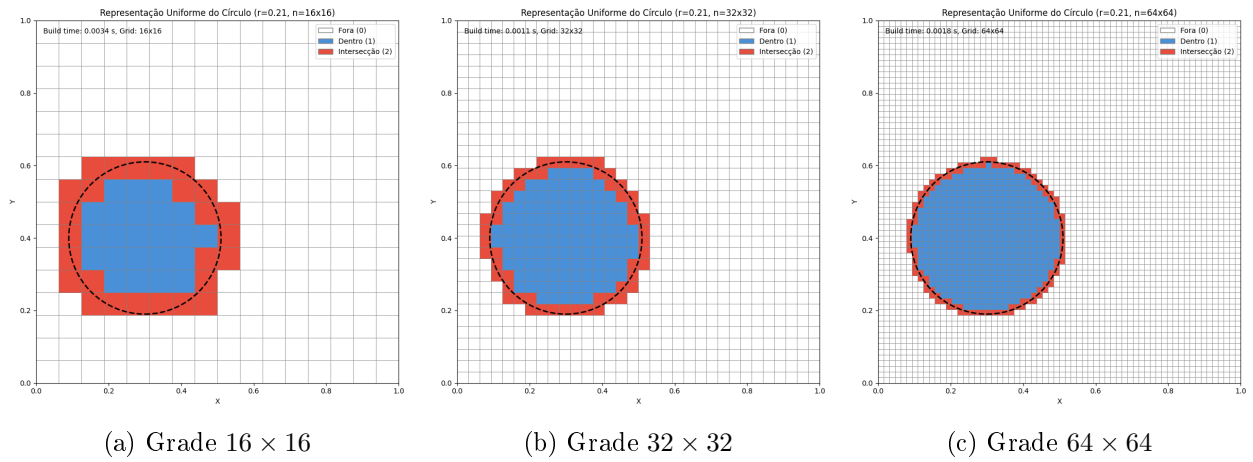


Figure 1: Representação Uniforme do Disco (centro $(0.3, 0.4)$, raio 0.21).

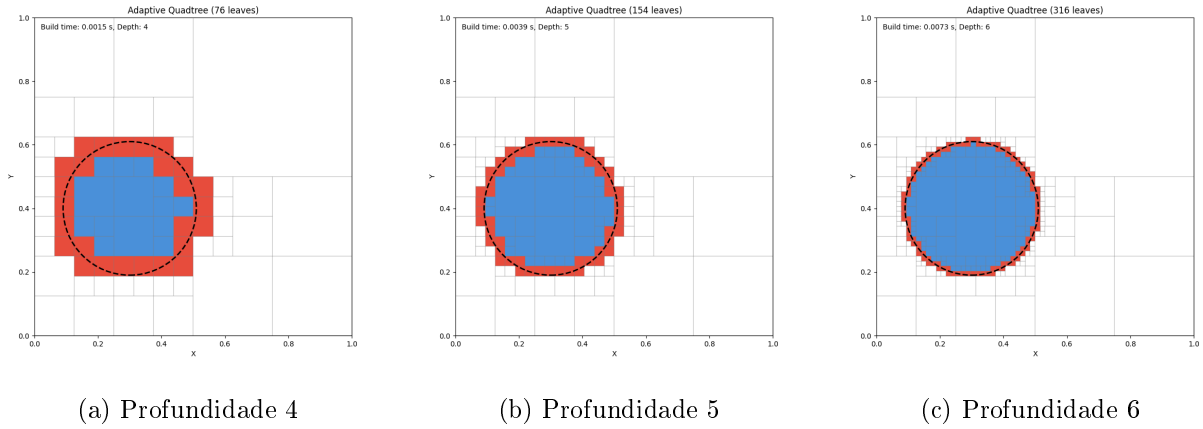


Figure 2: Representação Adaptativa (Quadtree) do Disco.

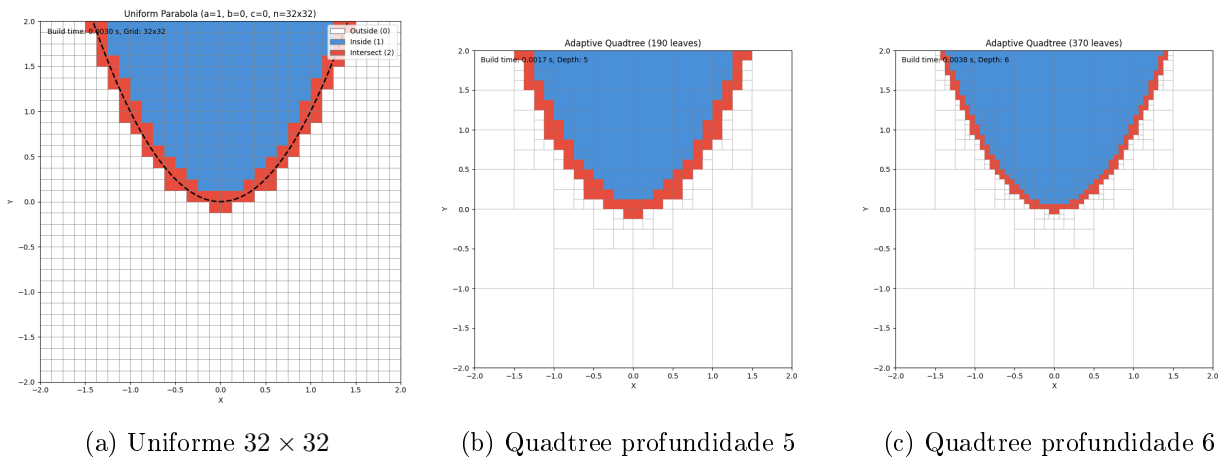


Figure 3: Parábola $y \geq x^2$: representações uniforme e adaptativa.

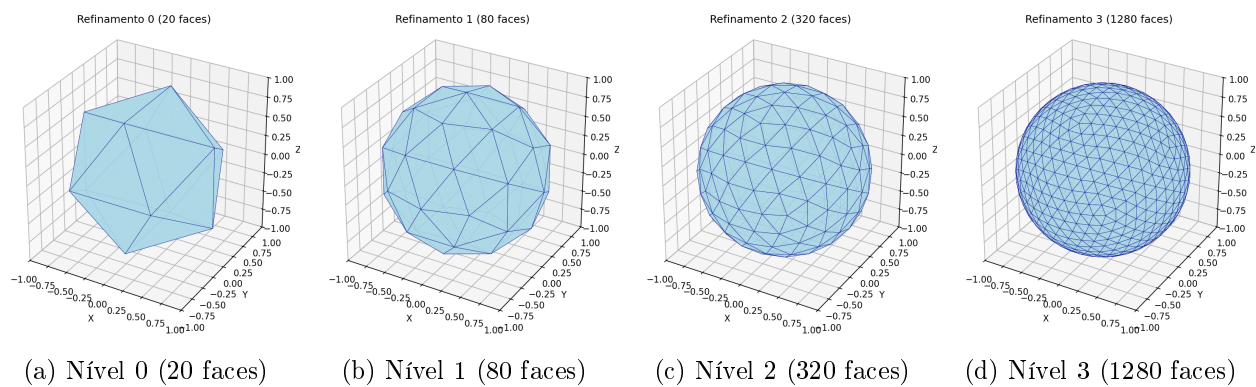


Figure 4: Refinamento da Esfera Geodésica: do icosaedro (nível 0) até nível 3. Cada nível quadruplica o número de faces.